

Staff Backend Engineer Test Task: Hybrid Search Engine & Intelligent Ingestion Pipeline

Context:

Lecturio aims to address the shortage of healthcare providers by increasing access to quality medical education. A critical component of this is enabling students to find relevant practice questions not just by matching keywords, but by understanding the *intent* and *pedagogical level* of the content.

1. The Objective

You are tasked with designing and prototyping a backend system that ingests raw medical exam questions, enriches them using Generative AI, and exposes them via a search API that supports both lexical (keyword-based) and semantic (vector-based) retrieval.

As a Staff Engineer, we look to you to define the architecture, make high-level technology decisions, and demonstrate hands-on capability with a Proof of Concept (PoC).

2. The Data Domain

Assume the input data consists of a JSON stream of questions with the following schema:

- Title: String (e.g., "Cardiology: Patient Symptoms")
- Content: String (The question stem/scenario)
- Answers: Array of Objects (Each containing content and is_correct boolean)
- Explanation: String (Detailed reasoning for the correct answer)

3. Functional Requirements

1. Data Pipeline & AI Enrichment:

- Ingest questions from a mock source (e.g., a local JSON file).
- Use an LLM to enrich each question with metadata before indexing. Specifically:
 - Bloom's Taxonomy Level: Classify the cognitive level (e.g., Application, Analysis, Evaluation).
 - Prominent Keywords: Extract medical entities or concepts not explicitly present in the text but relevant to the topic.
 - Anything else you think would make sense.

2. Hybrid Search:

- The search engine must support queries that rely on exact and fuzzy matches (e.g., specific drug names) and semantic meaning (e.g., "questions about heart failure symptoms").

3. Search API:

- A simple REST or GraphQL endpoint to query the indexed data.

4. Deliverables

Please provide the following in a GitHub repository or a zipped folder.

A. Architecture & Design (Documentation)

Demonstrates Technical Leadership & Architecture

- Architectural Diagram: High-level view of the system, including the ingestion pipeline, AI processing services, database/index storage, and API layer.
- Data Flow Diagram: Visualizing how a single question moves from "Raw" to "Index-Ready," including the asynchronous nature of the AI enrichment.
- Technology Short-list & Decision Record:
 - Propose 2-3 solutions for the search engine (e.g., OpenSearch vs. Pinecone vs. Postgres, etc.).
 - Analyze the Pros and Cons of each regarding scalability, maintenance, and cost.
 - Select one for the PoC and justify your choice. Note: We currently use AWS, OpenSearch, and Pinecone, but you are free to choose what fits best.

B. Strategic Planning

Demonstrates Strategic Planning & Project Management

- Plan of Action: Create a roadmap breaking this project into milestones.
- Risk Assessment: Briefly mention potential bottlenecks (e.g., LLM latency, cost, index drift) and how you would mitigate them.

C. Proof of Concept (Code)

Demonstrates Hands-on capability & AI Integration

Write a basic implementation of the system.

- Stack: TypeScript is preferred.
- Scope:
 - A script/service that reads a sample JSON file (5-10 mock questions).
 - An integration with an LLM (OpenAI, Anthropic, or local) to generate the Bloom's Taxonomy and Keyword labels.
 - Indexing of the enriched data into your chosen search solution.
 - A simple web application in which a user can chat with an agent and the agent can retrieve questions from the DB.

5. Evaluation Criteria

We are not looking for "perfect" production code, but rather "Staff-level" thinking. We will evaluate:

1. Architectural Clarity: Can you design systems that are scalable and maintainable?
2. Handling Ambiguity: How do you handle the "messy" problem of LLM non-determinism (e.g., output schema validation)?
3. AI Competency: Familiarity with prompting, JSON enforcement, or agentic frameworks (like Mastra or LangChain) is a plus.
4. Communication: Are your diagrams and written plans clear enough to align cross-functional teams?

Timebox Recommendation:

Please spend no more than **8 hours** on this task. We value your time; if you cannot finish a component, document what you *would* have done. *(Please track and report approximate time spent.)*

Considerations and Questions

If anything is unclear, feel free to ask. Clarifying questions are welcome and won't count against you.

Feel free to generate a dummy JSON file with questions.

If you need an OpenAI key we can provide you one.

Good luck, and we look forward to seeing your result!